GRK Test 1 · quick reference

lectures 1–6 · Szymański · Test Fri 13 Nov 2026 · 15 pts · conventions checked against the slides

Answer protocol (each task)

1. **Formula** first (partial credit). **2.** Table of steps. **3.** Drawing on the grid. **4.** Boxed answer with units.
2. State every **convention** where a choice exists: MB vs MiB, floor vs round, seed + fill order, pixel centres, clip order.
3. Read pixel coordinates from the axis labels **twice**. Do the fast calculations first.

L1 · Displays, pipeline, vectors

Data rate = W·H·(bytes/px)·Hz. Slide 26: 4K = 4096×2160 ≈ 10^7 ticks (with blanking) → $10^7 \cdot 3 \cdot 60$ = **$1.8 \cdot 10^9$ B/s ≈ 1.7 GiB/s**; pixels only $1.59 \cdot 10^9$.

Time per pixel = 1/(W·H·Hz). 1920×1080 @80 Hz → 1/165 888 000 = **6.03 ns** (frame 12.5 ms).

Vector memory (float 4 B): circle x,y,r = 3 → 12 B · triangle 6 → 24 B · rectangle 8 → 32 B · cube 24 → 96 B. 100 circles + 100 triangles = **3600 B**.

Pipeline (slide 30): vertex shader → triangle assembly (+ clip/cull) → **rasterization** → fragment shader → testing & blending (depth, alpha) → framebuffer. Programmable: vertex + fragment.

Fragment shader: IN interpolated data for one pixel (colour, UV, normal, depth/position) + uniforms (textures, lights). OUT colour RGBA (+ depth). Runs per fragment → most expensive → cull early.

Vertex shader: IN vertex attributes + matrices; OUT transformed position + data for interpolation.

Raster: refresh whole frame row by row (≥ 50–60 Hz, else flicker). Vector display: time depends on the drawing. Double buffer = draw in back, show front, swap. GPU = SIMD.

Vectors: AB = B − A. Sum tip-to-tail. $a \cdot b = \Sigma a_i b_i = |a||b|\cos\theta$. $a \times b$ = [a_y b_z − a_z b_y, a_z b_x − a_x b_z, a_x b_y − a_y b_x]; x×y = z, y×z = x, z×x = y; b×a = −a×b; |a×b| = area. Right-handed: x right, y up, z toward you. [−2,0,0]×[0,1,0] = **[0,0,−2]**. 3I·[2,−1,0] = [6,−3,0].

L2 · Image representation

Sampling = where (grid); quantization = which values (levels). A pixel is a sample: position + value, no size. Pixel resolution W×H; linear resolution ppi = px / inch (1″ = 2.54 cm).

Memory = W·H·bits/8 B; MB = 10^6 B (slide 14 uses decimal), MiB = 1 048 576 B. 1024×768×24 = 18 874 368 bit = 2 359 296 B = **2.36 MB** (2.25 MiB).

ppi: width = $d \cdot a/\sqrt{(a^2+b^2)}$; 16:9 → ×0.8716, height ×0.4903. 42″ 1920×1080 → 36.61″ → **52.45 ppi**.

Eye limit 1′ = 1/60°: x = D·tan 1′ (tan 1′ = 0.000 290 9; slide 0.00029), n = W/x. 30 cm @ 50 cm → x = 0.01454 cm → **2063** (slide: 0.0145 → 2069).

Bilinear (A B top, C D bottom; α across, β down):
P1 = αB + (1−α)A
P2 = αD + (1−α)C
N = βP2 + (1−β)P1
A30 B50 E45 F60, α=β=0.75 → 45, 56.25, **53.44**

2:1 down: new[y][x] = avg of old[2y..2y+1][2x..2x+1] (per channel). Up 1:2: copy or interpolate. Detail finer than the new pixel is lost (undersampling → aliasing, moiré); Q27: every 2×2 block → same 75 % grey.

L3 · Colour

Visible 380–780 nm. Eye: ~100 grey levels, ~10 M colours. 24 bit = 16 777 216.

Colours = $(2^b)^3$: 1→8, 2→64, **3→512**, 4→4096, 5→32 768, 6→262 144, 8→16.7 M.

8→n bits: q = floor(v / 2^(8−n)) (drop low bits; state it). Alt: round(v·(2^n−1)/255). 143,240,128 → 4 bit: **8, 15, 8** (143 & 128 merge). [242,254,4] → 3 bit [7,7,0].

CMY = 255 − RGB ([1,1,1] − RGB). [255,135,0] → [0,120,255]; black ↔ white. K = min(C,M,Y). Subtractive (ink); RGB additive (light).

Grey = 0.299R + 0.587G + 0.114B (also (R+G+B)/3, mind the brackets). [128,0,128] → **53**; [255,255,0] → **226** (average: 85, 170).

Mix C = $\alpha C_1 + (1-\alpha)C_2$; equal: average. [255,255,0]+[0,255,255] → **[128,255,128]**.

RGB cube: black (0,0,0) origin, white (1,1,1), R (1,0,0), G (0,1,0), B (0,0,1), Y (1,1,0), C (0,1,1), M (1,0,1); greys on the diagonal. Used by monitors, cameras, scanners.

HSV: V = max, S = (max−min)/max, H: max=R → 60·((G−B)/Δ mod 6); G → 60·((B−R)/Δ+2); B → 60·((R−G)/Δ+4). R0° Y60° G120° C180° B240° M300°. Cone: tip = black, axis = greys, rim of top disc = pure bright colours. [7,7,0] → H60 S1 V1 (rim); [7,0,7] → H300 (rim).

CIE xy: horseshoe; spectral (monochromatic) colours on the curved edge (nm), purple line straight; white ≈ (⅓,⅓). Two colours mix to **any point on the segment** between them; three → triangle (= device gamut). No 3 real primaries cover everything → XYZ uses imaginary primaries.

L4–5 · Rasterization

DDA (|m| ≤ 1, $x_0 < x_1$): m = Δy/Δx; y += m; plot (x, round y). (0,0)→(5,3): (0,0)(1,1)(2,1)(3,2)(4,2)(5,3). |m| > 1 → gaps ((0,0)→(2,5): (0,0)(1,3)(2,5)) → step y, x += 1/m.

> **Bresenham** (0 ≤ m ≤ 1, start at the smaller x): a = 2Δy, b = 2Δy − 2Δx, p_0 = 2Δy − Δx.
> p < 0 → (x+1, y), p += a · p ≥ 0 (incl. 0) → (x+1, y+1), p += b.
> (0,0)→(5,2): a=4, b=−6, p = −1, 3, −3, 1, −5 → (0,0)(1,0)(2,1)(3,1)(4,2)(5,2). Slide (2,2)→(8,5): a=6, b=−6, p_0=0 → up.

Steep: swap x↔y. Negative slope: |Δy|, step y−1. p = Δx(d_1−d_2) → integers only.

Antialiasing: per column the 2 pixels nearest the true line, intensity = 1 − distance, sum 100 %. (5,2) line: col 1: 60/40, 2: 20/80, 3: 80/20, 4: 40/60. Cannot remove aliasing, only reduce. FSAA = render bigger, average down.

> **Midpoint circle**: start (0,r), p_0 = 5/4 − r (1 − r same pixels).
> p < 0 → (x+1, y), p += $2x_+$ + 1 · p ≥ 0 → (x+1, y−1), p += $2x_+$ + 1 − $2y_+$ (new x, y). Stop x ≥ y; mirror (±x,±y), (±y,±x).
> r = 10: (0,10)(1,10)(2,10)(3,10)(4,9)(5,9)(6,8)(7,7) → 56 px; p: −8.75, −5.75, −0.75, 6.25, −2.75, 8.25, 5.25.

Scan-line: per row: crossings with edges x = x_1 + (y−y_1) (x_2−x_1)/(y_2−y_1) → sort → fill pairs 1–2, 3–4. Vertex: edges on opposite sides → count once; both same side (peak/valley) → twice. Horizontal edges: contour, not crossings. Say whether you fill the outline pixels.

Seed fill (slide 30): `if pixel==WHITE {paint; f(x−1,y); f(x+1,y); f(x,y+1); f(x,y−1)}` = left, right, up, down, 4-connected, depth-first. Name seed + order. Q23 seed (1,2): (1,2)(2,2)(2,3)(1,3)(3,3)(4,3)(4,4)(3,4)(4,2)(4,1). (Slide picture numbers row by row = "active list" version; state yours.)

> **Barycentric**: U = αA + βB + γC, α+β+γ = 1; inside ⇔ all ≥ 0; f(U) = αf_A + βf_B + γf_C (grey, RGB, UV, depth).
> d = (y_B−y_C)(x_A−x_C) + (x_C−x_B)(y_A−y_C); α = [(y_B−y_C)(x−x_C) + (x_C−x_B)(y−y_C)]/d; β = [(y_C−y_A)(x−x_C) + (x_A−x_C)(y−y_C)]/d; γ = 1−α−β.
> A(0,0) B(0,4) C(4,0), pixel **(1,2)**: γ = x/4 = .25, β = y/4 = .5, α = .25 → inside; grey .25·255 + .5·0 + .25·128 = **95.75 ≈ 96**.

L6 · 2D transforms

Column vectors $[x,y,1]^T$, $P' = M \cdot P$. T then S then R → **M = R·S·T** (first = rightmost).

$$\begin{vmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{vmatrix}_T \quad \begin{vmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{vmatrix}_S \quad \begin{vmatrix} \cos & -\sin & 0 \\ \sin & \cos & 0 \\ 0 & 0 & 1 \end{vmatrix} \; R(\theta) \text{ CCW}$$

About p: T(p)·X·T(−p). Scale about p: last column p(1−s). Rotate about (x_1,y_1): last column $x_1(1-\cos)+y_1 \sin$, $y_1(1-\cos)-x_1 \sin$. Mirror x-axis S(1,−1); shear [1 k 0; 0 1 0; 0 0 1].

T[1,−2] then S(.5,1): $M = \begin{vmatrix} 0.5 & 0 & 0.5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{vmatrix}$ → (1,−1)(2,−1)(1,1). Scale (1.5,.5) about (−3,2): M = [1.5 0 1.5; 0 .5 1] → (−6,1)(0,1)(0,3).

Frame change (slide 11): $[u\ v\ 1]^T$ = [x_u y_u 0; x_v y_v 0; 0 0 1]·T(−e)·$[x\ y\ 1]^T$, rows = new axes. Frame rotated θ = point rotated −θ. P(2,2), frame +90° → **(2,−2)**.

L6 · Clipping

Cohen–Sutherland code = [Top Bottom Right Left]: 1001 1000 1010 / 0001 0000 0010 / 0101 0100 0110. Both 0000 → accept; AND ≠ 0000 → reject; else cut an outside end at an edge (slide order left, top, right, bottom), recode, repeat. Cut x = c: $y = y_1 + (y_2-y_1)(c-x_1)/(x_2-x_1)$; cut y = c: $x = x_1 + (x_2-x_1)(c-y_1)/(y_2-y_1)$.

Window (4,0)–(10,5): (5,1)–(8,3) accept · (2,7)–(6,7) 1001&1000 → reject · (8,10)–(15,3) 1000/0010 → top (13,5) 0010 → reject.

Sutherland–Hodgman: one window edge at a time; polygon edges S→W (clockwise): out→in: P, W · in→in: W · in→out: P · out→out: nothing. Triangle (7,1)(1,4)(5,10) → **(4,2.5)(4,5)(6.11,5)(7,1)** (55/9 = 6.11).

L6 · Bézier

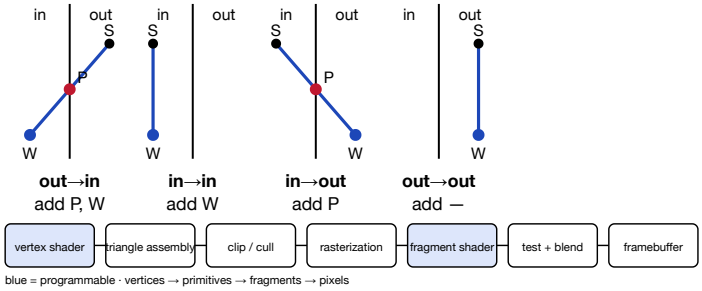
Degree n = points − 1. Quadratic $(1-t)^2P_0 + 2t(1-t)P_1 + t^2P_2$; cubic $(1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3$. Passes P_0 and P_n only; tangent to first/last segment; inside the convex hull.

de Casteljau: on each segment mark the point at t, join, repeat → B(t). (1,1)(4,8)(10,1): .25 → (2.69,3.63); .5 → **(4.75,4.5)**; .75 → (7.19,3.63). Cubic (1,1)(3,−7)(8,−7)(10,1): (2.97,−3.5) (5.5,−5) (8.03,−3.5).

Join: C0 $Q_0 = P_n$ · G1 P_{n-1}, P_n, Q_1 collinear · C1 $Q_1 = 2P_n - P_{n-1}$ (e.g. (12,9)).

Pictures to copy into answers

1001	1000	1010
0001	0000	0010
0101	0100	0110



Theory one-liners (true/false traps)

- A pixel has no size: it is a sample (position + value); the display gives it a size.
- Aliasing (jaggies) can only be **reduced**, never removed (finite raster).
- Raster refresh time does not depend on the picture; vector display time does.
- Frame buffer → CRT needs a DAC (digital → analogue). LCD/CRT use RGB, not CMY.
- Vertex shader processes vertices; fragment shader colours pixels. GPU = many ALUs in parallel (SIMD).
- More pixels ⇒ more memory; lower resolution ⇒ less memory.
- RGB is additive (light, black + light = colour); CMY(K) is subtractive (ink on white paper). K because C+M+Y gives muddy brown.
- RGB cannot show every visible colour (gamut = triangle inside the horseshoe).
- Spectral (monochromatic) colours lie on the curved edge of the chromaticity diagram; purples are not spectral.
- Metamers: different spectra seen as the same colour.
- Bresenham uses only integer additions; DDA uses floating point.
- Two crossing digital lines may share 0, 1 or more pixels (or none at all where they "cross" between pixels).
- Barycentric weights interpolate colour, depth and texture coordinates (Gouraud = barycentric colour).
- Homogeneous coords make translation a matrix multiplication; rotation centre need not be the origin ($T \cdot R \cdot T^{-1}$).
- Matrix products are not commutative: T·R ≠ R·T in general; two rotations about the same point commute.
- Cohen–Sutherland: AND = 0000 does not mean visible; a segment with both ends outside can still cross the window.
- Sutherland–Hodgman works for any polygon against a convex window; concave subjects may leave edges along the border.
- Bézier: degree = points − 1, global control (moving one point changes the whole curve), affine invariant, in the convex hull.
- Implicit f(x,y)=0 tests points; parametric (x,y)=f(t) generates points (used in graphics).

Wrong answers circulating (a 2025 student guide "Studying Graphics"): pixel time "6 µs" → **6 ns** · 3 bits/ch "8 colours" → **512** · circle p ≥ 0 "y+1" → **y−1** · frame change "R·P = (−2,2)" → **(2,−2)** · Bézier with P1 = (10,1) → control point is **P1 = (4,8)** · backface n_z "−5" → **13** (faces the viewer). Grey: it defaults to the average, so write **both** (53 weighted / 85 average).

Traps: bits vs bytes · MB vs MiB · $(2^b)^3$ not 2^b · p = 0 goes UP · start Bresenham at smaller x · circle update uses NEW x,y · mirror 8 ways · AND = 0 ≠ accept · matrix order (first = rightmost) · sin sign · frame = rotate point by −θ · pixel (1,2) not (1,1) · 55/9 not 6.

Part B · every task type, solved like a model answer

numbers from the tested solver · official example questions 2024 + Exam 1 2020 + EXAM 1/22, 1/23

Displays and image

Data rate (P1 Q1)

4K at 60 Hz, 24-bit colour.

1. pixels per frame = 4096 × 2160 = 8,847,360
2. bytes per pixel = 24 bits ÷ 8 = 3 B
3. data rate = pixels × bytes/px × refresh = 8,847,360 × 3 × 60 Hz = 1,592,524,800 B/s
4. in bits/s: × 8 = 1.274·10^10 b/s
5. Lecture slide 26 takes 4K = 4096×2160 and rounds up to 10^7 pixel-clock ticks (blanking included): 10^7 × 3 B × 60 = $1.8 \cdot 10^9$ B/s ≈ "1,7 GB/s" (binary GiB). Write the exact pixel count and mention that rounding.

Answer: slide: $1.8 \cdot 10^9$ B/s ≈ 1.7 GiB/s (exact pixels $1.59 \cdot 10^9$ B/s)

Time per pixel (P1 Q2)

1920×1080 at 80 Hz.

1. pixels per second = 1920 × 1080 × 80 = 165,888,000
2. time per pixel = 1 ÷ 165,888,000 s = 6.028·10^-9 s = 6.0282 ns
3. (per frame: 1/80 s = 12.5 ms; per line: 11.5741 µs)

Answer: ≈ 6.03 ns

Vector memory (P1 Q3)

100 circles, 100 triangles, 4-byte floats.

shape	count	numbers each	bytes each	subtotal
`circle`	`100`	`3`	`3·4 = 12 B`	`1,200 B`
`triangle`	`100`	`6`	`6·4 = 24 B`	`2,400 B`

Answer: 3,600 B

Bitmap memory (P1 Q6)

1024 × 768 × 24 in MB.

1. bits = 1024 × 768 × 24 = 18,874,368 bits
2. bytes = 18,874,368 ÷ 8 = 2,359,296 B
3. MB (10^6) = 2.3593 MB; MiB (2^{20} = 1,048,576) = 2.25 MiB; KB = 2359.3 kB / 2304 KiB
4. Write which megabyte you mean. Both are accepted if you say it.

Answer: 2,359,296 B = 2.3593 MB (2.25 MiB)

Linear resolution (P1 Q7)

42″, 1920×1080, 16:9.

1. aspect 16:9 → diagonal units = $\sqrt{(16^2 + 9^2)} = \sqrt{337} = 18.3576$
2. width = 42 · 16/18.3576 = 36.6062″, height = 42 · 9/18.3576 = 20.591″
3. linear resolution = 1920 px ÷ 36.6062″ = 52.45 ppi (check: 1080 ÷ 20.591 = 52.45 ppi)
4. pixel size = 25.4 mm ÷ 52.45 = 0.484 mm

Answer: 52.45 ppi (width 36.606″, height 20.591″)

Eye limit (P1 Q8)

Width 30 cm, distance 50 cm.

1. The eye separates details down to about 1 arc-minute = 1/60°.
2. smallest detail at distance D: x = D · tan(1/60°) = 50 · 0.0002909 = 0.01454 (same unit as D)
3. max pixels across the width: n = W / x = 30 / 0.01454 = 2062.6
4. Slide 20 rounds tan 1′ ≈ 0.00029: x = 50·0.00029 = 0.0145 → n = 2069 (the lecturer's number for 30 cm @ 50 cm is ≈ 2069). Show the formula; either value is fine.
5. More pixels than that are wasted: the eye cannot resolve them at this distance.

Answer: ≈ 2063 px (slide rounding: 2069)

Bilinear (P1 Q16)

A=30, B=50, E=45, F=60; N at ¾ across and ¾ down.

1. Layout: A —— B on the first row, C —— D on the second; α = horizontal fraction from A toward B, β = vertical fraction from row AB toward row CD.
2. P1 = α·B + (1−α)·A = 0.75·50 + 0.25·30 = 45
3. P2 = α·D + (1−α)·C = 0.75·60 + 0.25·45 = 56.25
4. N = β·P2 + (1−β)·P1 = 0.75·56.25 + 0.25·45 = 53.4375
5. Check: N must lie between the smallest and largest of A, B, C, D.

Answer: N = 53.4375 (≈ 53)

2:1 pseudo-code (P1 Q9)

Color[][] image = new Color[100][100]

```
Color[][] half(Color[][] img) {
  Color[][] out = new Color[50][50];
  for (int y = 0; y < 50; y++)
    for (int x = 0; x < 50; x++) {
      Color a = img[2*y][2*x],   b = img[2*y][2*x+1];
      Color c = img[2*y+1][2*x], d = img[2*y+1][2*x+1];
      out[y][x] = new Color((a.r+b.r+c.r+d.r)/4,
                            (a.g+b.g+c.g+d.g)/4,
                            (a.b+b.b+c.b+d.b)/4);
    }
  return out;
}
```

Simpler (nearest neighbour): out[y][x] = img[2y][2x] (faster, loses information, more aliasing).

Increase 1:2 (nearest neighbour): `big = new Color[200][200]; for y,x in 0..199: big[y][x] = img[y/2][x/2]` (integer division: each pixel becomes a 2×2 block; bilinear gives a smoother result).

Vectors (P1 Q28–31)

X = [−2,0,0], Y = [0,1,0] from P(−1,1,0).

X + Y = [−2,1,0] (tip at (−3,2,0)). a × b = [a_y b_z − a_z b_y, a_z b_x − a_x b_z, a_x b_y − a_y b_x] = [0·0 − 0·1, 0·0 − -2·0, -2·1 − 0·0] = [0, 0, -2]

Right-hand check: −x × y = −z → points away from you. 3I·[2,−1,0] = [6,−3,0] = scale ×3.

Colour

Reduce depth (P1 Q11)

Grey 143, 240, 128 at 8 bit → 4 bit.

8-bit value	÷ 2^4	4-bit (floor, main)	4-bit (round v·15/255)	floor shown back as 8-bit
143	143 / 16 = 8.938	8	8	136
240	240 / 16 = 15	15	14	255
128	128 / 16 = 8	8	8	136

Circle (143) and background (128) both become 8 → they merge; the object disappears.

3 bits/channel + count + HSV (EXAM 1/23 #1)

Circle [242,254,4], triangle [240,0,241], rectangle [247,13,253].

object	8-bit	3-bit (÷32)	HSV
circle	[242,254,4]	[7,7,0]	yellow H 60° S 1 V 1 → rim of top disc
triangle	[240,0,241]	[7,0,7]	magenta H 300° S 1 V 1 → rim
rectangle	[247,13,253]	[7,0,7]	magenta → same point: merges with triangle

Count: $(2^3)^3$ = **512** colours.

RGB → CMY (P1 Q12)

[255,135,0], [0,0,0], [255,255,255]

C = 255 − R etc. → **[0,120,255]**, **[255,255,255]**, **[0,0,0]**.

RGB → grey (P1 Q13)

[128,0,128], [255,255,0]

Y = 0.299·R + 0.587·G + 0.114·B = 0.299·128 + 0.587·0 + 0.114·128 = 38.272 + 0 + 14.592 = 52.864 → 53

Y = 0.299·R + 0.587·G + 0.114·B = 0.299·255 + 0.587·255 + 0.114·0 = 76.245 + 149.685 + 0 = 225.93 → 226

Mix (P1 Q15)

A = [255,255,0], B = [0,255,255], equal parts.

1. mix in proportion 0.5 : 0.5 → each channel = 0.5·c1 + 0.5·c2
2. R = 0.5·255 + 0.5·0 = 127.5; G = 255; B = 127.5
3. On the chromaticity diagram the mix lies on the segment between the two colours (closer to the bigger share).
4. (Adding light at full power instead: clamp(c1 + c2) = [255, 255, 255].)

Answer: [128, 255, 128] (exact [127.5, 255, 127.5])

Chromaticity (P1 Q14)

Colours 1 and 2 on the xy diagram.

Every colour on the straight segment 1–2 (and only those); more of colour 1 → closer to 1. Draw the segment.

Describe RGB (P1 Q17)

Components, axes, white/black/yellow, uses.

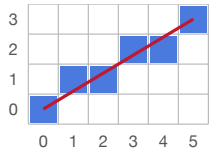
Additive model: light of red, green, blue. Unit cube with axes R, G, B; black (0,0,0) at the origin, white (1,1,1) opposite, yellow (1,1,0) = R + G (8-bit: [255,255,0]). Greys on the black–white diagonal. Used by emissive devices: monitors, TVs, projectors, cameras, scanners.

Lines

DDA (P1 Q18)

(0,0) → (5,3)

x	y exact	round(y)	pixel
0	0	0	(0,0)
1	0.6	1	(1,1)
2	1.2	1	(2,1)
3	1.8	2	(3,2)
4	2.4	2	(4,2)
5	3	3	(5,3)



Why slope ≤ 45° (P1 Q19)

Example (0,0) → (2,5)

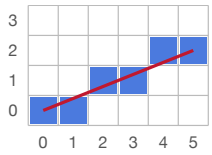
Stepping x gives y = 0, 2.5, 5 → only (0,0), (1,3), (2,5): rows 1, 2, 4 empty = gaps. For |m| > 1 step y by 1, add 1/m to x: (0,0)(0,1)(1,2)(1,3)(2,4)(2,5).

Bresenham (P1 Q20)

(0,0) → (5,2)

Δx = 5, Δy = 2; a = 2Δy = 4, b = 2Δy − 2Δx = -6, p_0 = 2Δy − Δx = -1

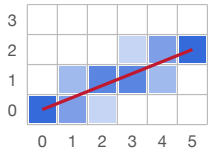
current	p	decision	next	new p
(0,0)	−1	p < 0 → straight	(1,0)	−1 + a = 3
(1,0)	3	p ≥ 0 → step (diagonal)	(2,1)	3 + b = −3
(2,1)	−3	p < 0 → straight	(3,1)	−3 + a = 1
(3,1)	1	p ≥ 0 → step (diagonal)	(4,2)	1 + b = −5
(4,2)	−5	p < 0 → straight	(5,2)	(end)



Antialiasing (P1 Q25)

Antialiased version of the Q20 line.

x	true y	lower pixel	upper pixel
0	0	(0,0) 100 %	–
1	0.4	(1,0) 60 %	(1,1) 40 %
2	0.8	(2,0) 20 %	(2,1) 80 %
3	1.2	(3,1) 80 %	(3,2) 20 %
4	1.6	(4,1) 40 %	(4,2) 60 %
5	2	(5,2) 100 %	–



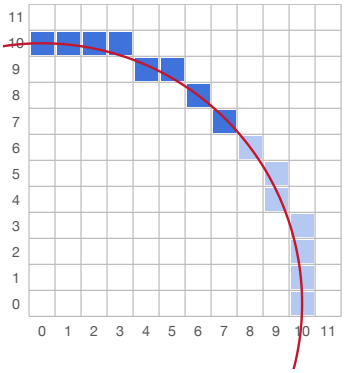
Principle: intensity by distance to the true line, constant sum per column; aliasing is reduced, not removed.

Circle, fills, triangles

Midpoint circle (P1 Q21)

Centre (0,0), r = 10.

current	p	decision	next	new p
(0,10)	−8.75	p < 0 → same row	(1,10)	−8.75 + 2·1 + 1 = −5.75
(1,10)	−5.75	p < 0 → same row	(2,10)	−5.75 + 2·2 + 1 = −0.75
(2,10)	−0.75	p < 0 → same row	(3,10)	−0.75 + 2·3 + 1 = 6.25
(3,10)	6.25	p ≥ 0 → one row down	(4,9)	6.25 + 2·4 + 1 − 2·9 = −2.75
(4,9)	−2.75	p < 0 → same row	(5,9)	−2.75 + 2·5 + 1 = 8.25
(5,9)	8.25	p ≥ 0 → one row down	(6,8)	8.25 + 2·6 + 1 − 2·8 = 5.25
(6,8)	5.25	p ≥ 0 → one row down	(7,7)	5.25 + 2·7 + 1 − 2·7 = 6.25



Dark = computed octant, light = mirrored (first quadrant shown; 56 pixels in the full circle).

Scan-line (P1 Q22)

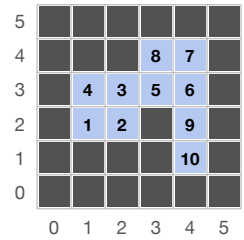
"Hill" outline: (0,4) (14,4) (14,5) (8,11) (4,11) (0,7).

row y	crossings	span incl. outline	interior pixels
11	–	4…8	outline row
10	3, 9	3…9	5
9	2, 10	2…10	7
8	1, 11	1…11	9
7	0, 12	0…12	11
6	0, 13	0…13	12
5	0, 14	0…14	13
4	0, 14	0…14	outline row

Rows 4 and 11 are horizontal outline edges (no crossings). Each other row: 2 crossings → fill strictly between them: 13, 12, 11, 9, 7, 5 pixels.

Seed fill order (P1 Q23)

Seed (1,2); order left, right, up, down (recursive).



From (1,2): left is border → right (2,2) → its right (3,2) is border, up (2,3) → left (1,3) dead end → back, right (3,3) → (4,3) → up (4,4) → left (3,4) → unwind to (4,3): down (4,2) → (4,1).

Barycentric inside + grey (P1 Q24, Q26)

A(0,0)=255, B(0,4)=0, C(4,0)=128, pixel (1,2).

1. U = αA + βB + γC with α + β + γ = 1 → three equations:
2. x: 1 = 0α + 0β + 4γ
3. y: 2 = 0α + 4β + 0γ
4. 1 = α + β + γ
5. → α = 0.25, β = 0.5, γ = 0.25 (check: sum = 1)
6. all > 0 → INSIDE
7. value = 0.25·255 + 0.5·0 + 0.25·128 = 95.75 ≈ 96

Answer: inside; grey ≈ 96

Texture colour (EXAM 1/22 #3)

(0,0)→uv(0,0), (7,0)→(1,0), (7,7)→(1,1); pixel (4,3).

→ α = 0.4286, β = 0.1429, γ = 0.4286 (check: sum = 1) all > 0 → INSIDE → (u,v) = (0.571, 0.429) = (4/7, 3/7) → texel column 0.4–0.6, row 0.4–0.6 → **(20, 20, 100)**.

2D transformations

Translate then scale (P2 Q1)

(1,1) (3,1) (1,3); T[1,−2], then S(0.5, 1).

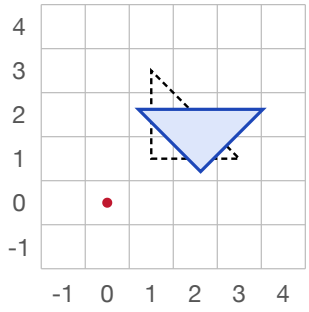
1. 1. T(1, -2) = [1, 0, 1 ; 0, 1, -2 ; 0, 0, 1]
2. 2. S(0.5, 1) = [0.5, 0, 0 ; 0, 1, 0 ; 0, 0, 1]
3. Composite (applied right-to-left): M = S(0.5, 1) · T(1, -2) = [0.5, 0, 0.5 ; 0, 1, -2 ; 0, 0, 1]
4. Column vectors $[x, y, 1]^T$: the FIRST transformation is the RIGHTMOST matrix. Order matters (T·R ≠ R·T).
5. M·$[1, 1, 1]^T$ = (0.5·1 + 0·1 + 0.5, 0·1 + 1·1 + -2) = (1, -1)
6. M·$[3, 1, 1]^T$ = (0.5·3 + 0·1 + 0.5, 0·3 + 1·1 + -2) = (2, -1)
7. M·$[1, 3, 1]^T$ = (0.5·1 + 0·3 + 0.5, 0·1 + 1·3 + -2) = (1, 1)

Answer: (1, -1), (2, -1), (1, 1)

Translate then rotate 45° (Exam 2020 T4, group 1)

(1,1) (3,1) (1,3); T[1,−2], then R(45°) about the origin.

1. 1. T(1, -2) = [1, 0, 1 ; 0, 1, -2 ; 0, 0, 1]
2. 2. R(45°) = [0.7071, -0.7071, 0 ; 0.7071, 0.7071, 0 ; 0, 0, 1]
3. Composite (applied right-to-left): M = R(45°) · T(1, -2) = [0.7071, -0.7071, 2.1213 ; 0.7071, 0.7071, -0.7071 ; 0, 0, 1]
4. Column vectors $[x, y, 1]^T$: the FIRST transformation is the RIGHTMOST matrix. Order matters (T·R ≠ R·T).
5. M·$[1, 1, 1]^T$ = (0.7071·1 + -0.7071·1 + 2.1213, 0.7071·1 + 0.7071·1 + -0.7071) = (2.1213, 0.7071)
6. M·$[3, 1, 1]^T$ = (0.7071·3 + -0.7071·1 + 2.1213, 0.7071·3 + 0.7071·1 + -0.7071) = (3.5355, 2.1213)
7. M·$[1, 3, 1]^T$ = (0.7071·1 + -0.7071·3 + 2.1213, 0.7071·1 + 0.7071·3 + -0.7071) = (0.7071, 2.1213)



Answer: (2.121, 0.707), (3.536, 2.121), (0.707, 2.121)

All four 2020 groups (T then R45°)

group	triangle, T	M (top two rows)	result
1	(1,1)(3,1) (1,3) T[1,−2]	[0.707 −0.707 2.121] [0.707 0.707 −0.707]	(2.121, 0.707) (3.536, 2.121) (0.707, 2.121)
2	(1,2)(3,2) (1,4) T[−1,2]	[0.707 −0.707 −2.121] [0.707 0.707 0.707]	(−2.828, 2.828) (−1.414, 4.243) (−4.243, 4.243)
3	(2,1)(4,1) (2,3) T[−1,−2]	[0.707 −0.707 0.707] [0.707 0.707 −2.121]	(1.414, 0) (2.828, 1.414) (0, 1.414)
4	(2,2)(4,2) (2,4) T[1,2]	[0.707 −0.707 −0.707] [0.707 0.707 2.121]	(−0.707, 4.95) (0.707, 6.364) (−2.121, 6.364)

Scale about a point (EXAM 1/23 #3)

(–5,0) (–1,0) (–1,4); S(1.5, 0.5) about (–3,2).

1. 1. S(1.5, 0.5) about (-3, 2) = T(-3,2)·S(1.5, 0.5)·T(3,-2) = [1.5, 0, 1.5 ; 0, 0.5, 1 ; 0, 0, 1]
2. Composite (applied right-to-left): M = S(1.5, 0.5) = [1.5, 0, 1.5 ; 0, 0.5, 1 ; 0, 0, 1]
3. Column vectors [x, y, 1]^T: the FIRST transformation is the RIGHTMOST matrix. Order matters (T·R ≠ R·T).
4. M·[-5, 0, 1]^T = (1.5·-5 + 0·0 + 1.5, 0·-5 + 0.5·0 + 1) = (-6, 1)
5. M·[-1, 0, 1]^T = (1.5·-1 + 0·0 + 1.5, 0·-1 + 0.5·0 + 1) = (0, 1)
6. M·[-1, 4, 1]^T = (1.5·-1 + 0·4 + 1.5, 0·-1 + 0.5·4 + 1) = (0, 3)

Answer: (-6, 1), (0, 1), (0, 3)

Frame change (P2 Q2)

P(2,2); EUV rotated 90° CCW.

1. Axes of the new frame (unit vectors, measured in OXY): u = [0, 1], v = [-1, 0] (frame rotated 90° CCW); origin E = (0, 0)
2. 1. move to the new origin: P − E = [2, 2]
3. 2. project on the axes: u = (P − E)·u = 2, v = (P − E)·v = -2
4. Matrix form: [u; v; 1] = [u_x u_y 0; v_x v_y 0; 0 0 1] · T(0, 0) · [x; y; 1] (rows = the new axes). Equivalently: rotate the POINT by −90°.

Answer: (u, v) = (2, -2)

Clipping

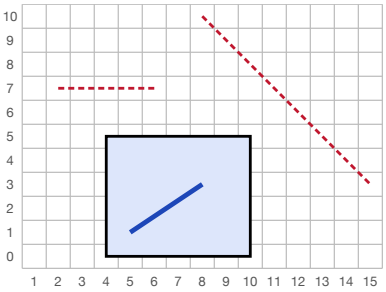
Cohen–Sutherland (P2 Q3)

Window (4,0)–(10,5). Segments (5,1)–(8,3), (2,7)–(6,7), (8,10)–(15,3).

Segment 1: (5,1)–(8,3)
P1(5,1) code 0000, P2(8,3) code 0000; AND = 0000
both 0000 → ACCEPT (5,1)–(8,3)

Segment 2: (2,7)–(6,7)
P1(2,7) code 1001, P2(6,7) code 1000; AND = 1000
AND = 1000 ≠ 0000 → REJECT (whole segment outside on one side)

Segment 3: (8,10)–(15,3)
P1(8,10) code 1000, P2(15,3) code 0010; AND = 0000
AND = 0000 but not both 0000 → clip
clip P1 against top y = 5: x = $x_1 + (x_2–x_1)(y–y_1)/(y_2–y_1)$ → (13,5), new code 0010
new AND = 0010
AND = 0010 ≠ 0000 → REJECT (whole segment outside on one side)



Cohen–Sutherland, Exam 2020 groups 1–4

Same window.

group	segment 1	segment 2	segment 3
1	accept (4, 1.2)–(10, 2.4)	reject	reject
2	accept (4, 2.2)–(10, 3.4)	reject	reject
3	accept (4, 3.2)–(10, 3.8)	reject	reject
4	accept (4, 2.25)–(10, 3.75)	reject	reject

Sutherland–Hodgman (P2 Q4)

Triangle (7,1), (1,4), (5,10); window (4,0)–(10,5).

Clip against left x = 4

(5, 10)→(7, 1): in → in, output W (7, 1)
(7, 1)→(1, 4): in → out, output crossing P (4, 2.5)
(1, 4)→(5, 10): out → in, output P (4, 8.5) and W (5, 10)
→ (7, 1) (4, 2.5) (4, 8.5) (5, 10)

Clip against top y = 5

(5, 10)→(7, 1): out → in, output P (6.11, 5) and W (7, 1)
(7, 1)→(4, 2.5): in → in, output W (4, 2.5)
(4, 2.5)→(4, 8.5): in → out, output crossing P (4, 5)
(4, 8.5)→(5, 10): out → out, output nothing
→ (6.11, 5) (7, 1) (4, 2.5) (4, 5)

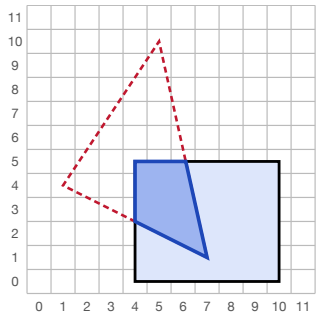
Clip against right x = 10

(4, 5)→(6.11, 5): in → in, output W (6.11, 5)
(6.11, 5)→(7, 1): in → in, output W (7, 1)
(7, 1)→(4, 2.5): in → in, output W (4, 2.5)
(4, 2.5)→(4, 5): in → in, output W (4, 5)
→ (6.11, 5) (7, 1) (4, 2.5) (4, 5)

Clip against bottom y = 0

(4, 5)→(6.11, 5): in → in, output W (6.11, 5)
(6.11, 5)→(7, 1): in → in, output W (7, 1)
(7, 1)→(4, 2.5): in → in, output W (4, 2.5)
(4, 2.5)→(4, 5): in → in, output W (4, 5)
→ (6.11, 5) (7, 1) (4, 2.5) (4, 5)

Answer: (6.11, 5), (7, 1), (4, 2.5), (4, 5) = the quadrilateral (4, 2.5), (4, 5), (6.11, 5), (7, 1)

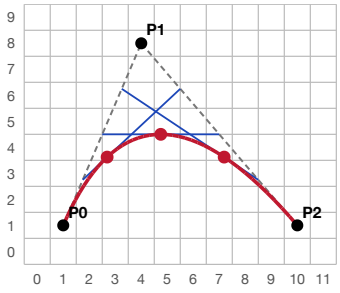


Bézier

Quadratic (P2 Q5)

P0 (1,1), P1 (4,8), P2 (10,1); t = 0.25, 0.5, 0.75.

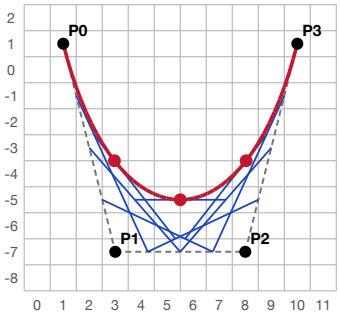
1. Degree 2 (quadratic = second order, 3 points). The curve starts at P0, ends at P2, and is pulled toward (but does not pass through) the inner control points.
2. Construction for a given t: on every segment of the control polygon mark the point at fraction t; join the new points and repeat until one point is left = B(t).
3. t = 0.25, round 1: (1.75, 2.75) (5.5, 6.25)
4. t = 0.25, round 2: (2.6875, 3.625)
5. t = 0.5, round 1: (2.5, 4.5) (7, 4.5)
6. t = 0.5, round 2: (4.75, 4.5)
7. t = 0.75, round 1: (3.25, 6.25) (8.5, 2.75)
8. t = 0.75, round 2: (7.1875, 3.625)
9. Formula: $B(t) = (1-t)^2P0 + 2t(1-t)P1 + t^2P2$
10. Smooth joining of a next curve Q: Q0 = P2 (10, 1) (C0, continuous), and Q1 on the line P1P2 beyond P2 (G1, same tangent direction). Equal length too (C1): Q1 = 2P2 − P1 = (16, -6).



Cubic + smooth join (EXAM 1/22 #1)

P0 (1,1), P1 (3,−7), P2 (8,−7), P3 (10,1).

1. Degree 3 (cubic = third order, 4 points). The curve starts at P0, ends at P3, and is pulled toward (but does not pass through) the inner control points.
2. Construction for a given t: on every segment of the control polygon mark the point at fraction t; join the new points and repeat until one point is left = B(t).
3. t = 0.25, round 1: (1.5, -1) (4.25, -7) (8.5, -5)
4. t = 0.25, round 2: (2.1875, -2.5) (5.3125, -6.5)
5. t = 0.25, round 3: (2.9688, -3.5)
6. t = 0.5, round 1: (2, -3) (5.5, -7) (9, -3)
7. t = 0.5, round 2: (3.75, -5) (7.25, -5)
8. t = 0.5, round 3: (5.5, -5)
9. t = 0.75, round 1: (2.5, -5) (6.75, -7) (9.5, -1)
10. t = 0.75, round 2: (5.6875, -6.5) (8.8125, -2.5)
11. t = 0.75, round 3: (8.0313, -3.5)
12. Formula: $B(t) = (1-t)^3P0 + 3t(1-t)^2P1 + 3t^2(1-t)P2 + t^3P3$
13. Smooth joining of a next curve Q: Q0 = P3 (10, 1) (C0, continuous), and Q1 on the line P2P3 beyond P3 (G1, same tangent direction). Equal length too (C1): Q1 = 2P3 − P2 = (12, 9).



Part C · working without a calculator

all values computed, rounded to 4 decimals

sin / cos / tan

θ	cos	sin	tan
0°	1	0	0
15°	0.9659	0.2588	0.2679
30°	0.866	0.5	0.5774
45°	0.7071	0.7071	1
60°	0.5	0.866	1.7321
75°	0.2588	0.9659	3.7321
90°	0	1	∞
120°	−0.5	0.866	−1.7321
135°	−0.7071	0.7071	−1
150°	−0.866	0.5	−0.5774
180°	−1	0	0
270°	0	−1	∞

√2/2 = 0.7071 · √3/2 = 0.8660 · tan 1′ = 0.000 290 9 · 1′ = 1/60° · 1″ = 2.54 cm

Bernstein weights, quadratic

t	$(1-t)^2$	$2t(1-t)$	t^2
0.25	0.5625	0.375	0.0625
0.5	0.25	0.5	0.25
0.75	0.0625	0.375	0.5625
0.333	0.4444	0.4444	0.1111
0.667	0.1111	0.4444	0.4444

Bernstein weights, cubic

t	$(1-t)^3$	$3t(1-t)^2$	$3t^2(1-t)$	t^3
0.25	0.4219	0.4219	0.1406	0.0156
0.5	0.125	0.375	0.375	0.125
0.75	0.0156	0.1406	0.4219	0.4219
0.333	0.2963	0.4444	0.2222	0.037
0.667	0.037	0.2222	0.4444	0.2963

Aspect ratio → width, height share of the diagonal

ratio	diag	width	height
16:9	√337 = 18.3576	0.8716	0.4903
4:3	√25 = 5	0.8	0.6
16:10	√356 = 18.868	0.848	0.53
21:9	√522 = 22.8473	0.9191	0.3939
5:4	√41 = 6.4031	0.7809	0.6247

Powers of 2

n	2^n
0	1
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1,024
11	2,048
12	4,096
16	65,536
20	1,048,576
24	16,777,216

Colours per bit depth

b / ch	levels	colours	bits/px
1	2	8	3
2	4	64	6
3	8	512	9
4	16	4,096	12
5	32	32,768	15
6	64	262,144	18
7	128	2,097,152	21
8	256	16,777,216	24

8 → 3 bits (floor v/32)

3-bit	8-bit range
0	0 – 31
1	32 – 63
2	64 – 95
3	96 – 127
4	128 – 159
5	160 – 191
6	192 – 223
7	224 – 255

8 → 4 bits (floor v/16)

4-bit	8-bit range
0	0 – 15
1	16 – 31
2	32 – 47
3	48 – 63
4	64 – 79
5	80 – 95
6	96 – 111
7	112 – 127
8	128 – 143
9	144 – 159
10	160 – 175
11	176 – 191
12	192 – 207
13	208 – 223
14	224 – 239
15	240 – 255

8 → 2 bits (floor v/64)

2-bit	8-bit range
0	0 – 63
1	64 – 127
2	128 – 191
3	192 – 255

Grey weights and CMY

v	.299v	.587v	.114v	255−v
0	0	0	0	255
16	4.78	9.39	1.82	239
32	9.57	18.78	3.65	223
64	19.14	37.57	7.3	191
96	28.7	56.35	10.94	159
128	38.27	75.14	14.59	127
135	40.36	79.24	15.39	120
160	47.84	93.92	18.24	95
192	57.41	112.7	21.89	63
200	59.8	117.4	22.8	55
224	66.98	131.49	25.54	31
240	71.76	140.88	27.36	15
255	76.24	149.69	29.07	0

Reciprocals

	value
1/2	0.5
1/3	0.33333
1/4	0.25
1/5	0.2
1/6	0.16667
1/7	0.14286
1/8	0.125
1/9	0.11111
1/10	0.1
1/11	0.09091
1/12	0.08333
1/16	0.0625
1/60	0.01667

Square roots

	value
√2	1.4142
√3	1.7321
√5	2.2361
√8	2.8284
√10	3.1623
√13	3.6056
√17	4.1231
√20	4.4721
√29	5.3852
√34	5.831
√41	6.4031
√45	6.7082
√50	7.0711
√52	7.2111
√58	7.6158
√61	7.8102
√65	8.0623
√73	8.544
√80	8.9443
√85	9.2195
√337	18.3576

Bytes

1 B = 8 bit · kB 10^3, MB 10^6, GB 10^9 · KiB 1024, MiB 1 048 576, GiB 1 073 741 824 · 1 ns = 10^{-9} s

Common resolutions

name	W×H	pixels
VGA	640×480	307 200
SVGA	800×600	480 000
XGA	1024×768	786 432
WXGA	1280×800	1 024 000
SXGA	1280×1024	1 310 720
WUXGA	1920×1200	2 304 000
HD	1280×720	921 600
Full HD	1920×1080	2 073 600
QHD	2560×1440	3 686 400
4K UHD	3840×2160	8 294 400
4K DCI (slide)	4096×2160	8 847 360
5K	5120×2880	14 745 600
8K	7680×4320	33 177 600

Part C2 · doing every task by hand

lecturer (Teams, 2 Oct): open book, own notes and books OK, **no electronic devices**, so no calculator

General tricks. Keep fractions until the last line (3/5, 5/7, ¾); only then convert with the table. Keep powers of ten apart ($a \cdot 10^n$). Divide by 8 early (24 bit = 3 B). For 45° keep $\sqrt{2}/2$ symbolic, then use k·0.7071. Check every result by size (is N between the corner values? is the pixel inside the grid?). Show the formula: method marks don't need the final digits.

Data rate of a display (P1 Q1): Keep powers of ten apart: 4096·2160 ≈ $8.85 \cdot 10^6$ (slide rounds to 10^7). Bytes × Hz first: 3·60 = 180. Then $10^7 \cdot 180 = 1.8 \cdot 10^9$ B/s. For 1920·1080 use the resolution table (2,073,600).

Time to display one pixel (P1 Q2): W·H from the resolution table, × Hz → write it as $a \cdot 10^n$ (1920·1080·80 = $1.65888 \cdot 10^8$). Then 1/a from the reciprocal table: 1/1.659 ≈ 0.603 → $0.603 \cdot 10^{-8}$ s = 6.03 ns.

Memory of a vector image (P1 Q3): All integers: numbers per shape × 4 B × count. Circle 12 B, triangle 24 B, rectangle 32 B, cube 96 B: just multiply by 100.

Vectors: components, sum, cross product, matrix × vector (P1 Q28–31): Integer arithmetic only. Cross product with the "xyzzy" pattern; check the direction with the right-hand rule. Matrix × vector = row · vector, one row at a time.

Memory of a bitmap (P1 Q6): Divide by 8 early: 24 bit = 3 B, 16 bit = 2 B. W·H from the resolution table, ×3. MB = move the comma 6 places (2,359,296 B = 2.36 MB).

Linear resolution (ppi) of a display (P1 Q7): width = diagonal × 0.8716 for 16:9 (×0.8 for 4:3; table). 42·0.8716: 42·0.87 = 36.54, +42·0.0016 = 0.07 → 36.61. ppi = 1920/36.61: 36.61·50 = 1830.5, rest 89.5 ≈ 2.45·36.61 → 52.45.

Maximum useful resolution (human eye) (P1 Q8): n = W/(D·tan1′) = (W/D)·3438 (1/tan 1′ = 3437.7). 30/50 = 0.6 → 0.6·3438 = 2063. With the slide's 0.00029: (W/D)·3448 = 2069.

Pseudo-code: decrease resolution 2:1 (P1 Q9): No numbers: write the 2×2 average loop. Mention the per-channel division by 4.

Resampling: intensity of a new pixel (bilinear) (P1 Q16): Use fractions, not decimals: α = ¾ → P1 = (A + 3B)/4; β = ¾ → N = (P1 + 3P2)/4. (30 + 150)/4 = 45; (45 + 180)/4 = 56.25; (45 + 168.75)/4 = 53.44.

Sketch the image resampled to lower ppi (P1 Q27): Count dark pixels in each 2×2 block: 0, 1, 2, 3, 4 dark → 0 %, 25 %, 50 %, 75 %, 100 % grey.

How many colours for b bits per channel (P1 Q10, EXAM 1/23): Powers-of-2 table: $(2^b)^3 = 2^{(3b)}$. 3 bits → $2^9 = 512$; 4 → $2^{12} = 4096$; 8 → $2^{24} = 16{,}777{,}216$.

Reduce colour/grey depth (8 → n bits) (P1 Q11, EXAM 1/23 #1): No division needed: find the value's bin in the "8 → n bits" table (÷32 for 3 bits: 0–31 → 0, 32–63 → 1 … 224–255 → 7; ÷16 for 4 bits).

RGB → CMY (P1 Q12): 255 − v: count up from v to 255 (135 → 255 is 120).

RGB → greyscale (P1 Q13): Weight table (0.299v, 0.587v, 0.114v) for round values, or approximate 0.3R + 0.59G + 0.11B (off by < 1). For 128: 0.299·128 = 38.27, 0.114·128 = 14.59 → 52.86. The average (R+G+B)/3 is easy: write both.

Chromaticity diagram: what can be mixed (P1 Q14): No numbers: draw the straight segment between 1 and 2.

Mix two RGB colours (P1 Q15): (a + b)/2 per channel; 255/2 = 127.5 → 128. For 25/75: (a + 3b)/4.

Describe the RGB model (P1 Q17): No numbers: draw the cube with the 8 vertices labelled.

Mark colours in the HSV cone (EXAM 1/23 #1c): V = max/255. S = (max − min)/max (1 if min = 0). H: pure mixes from the hue clock (R 0, Y 60, G 120, C 180, B 240, M 300); otherwise H = sector start ± 60·(mid − min)/(max − min). [255,128,0]: 60·128/255 ≈ 60·0.5 = 30°.

DDA: steps and pixels (P1 Q18): Keep m as a fraction (3/5): y = 0, 3/5, 6/5, 9/5, 12/5, 3 → round by comparing the remainder with half the denominator (3/5 ≥ ½ → up).

Why slope ≤ 45° in basic DDA (P1 Q19): No arithmetic: show 3 isolated pixels for (0,0)→(2,5).

Bresenham: steps and pixels (P1 Q20, Gracki tests): Integers only, made for no calculator: a, b, p_0, then add a or b each step.

Midpoint circle (P1 Q21): Use $p_0 = 1 - r$ (integer, same pixels as 5/4 − r): r = 10 → p = −9, −6, −1, 6, −3, 8, 5. Updates: $+2x_+ + 1$ or $+2x_+ + 1 - 2y_+$. Check: $x^2 + y^2 \approx r^2$.

Scan-line fill (P1 Q22, Gracki K1): Crossing $x = x_1 + (y - y_1) \cdot (\Delta x/\Delta y)$: Δx/Δy is a constant fraction per edge, so add it row by row.

Seed (starting-point) fill: order of pixels (P1 Q23): No arithmetic: number the pixels while following the recursion.

Barycentric: inside test and colour (P1 Q24, Q26; EXAM 1/22 #3 (texture)): Right triangle at the origin, A(0,0), B(0,h), C(w,0): β = y/h, γ = x/w, α = 1 − β − γ. Otherwise use the Cramer formula with fractions. Value: a weighted sum with quarters: .25·255 = 63.75, .25·128 = 32.

Antialiased version of a line (P1 Q25): Fractional parts of y = m·x: with m = 2/5 → .4, .8, .2, .6. Weights are 1 − f (lower pixel) and f (upper).

Compose 2D transformations and apply (P2 Q1, Exam 2020 T4 (all groups), EXAM 1/23 #3): Multiply matrices with integers first; keep $\sqrt{2}/2$ as a symbol for 45° and use the k·0.7071 table at the end: (1 − (−1))·$\sqrt{2}/2$ = 2·0.7071 = 1.414.

Change of coordinate frame (P2 Q2): 90° multiples need no trig: frame +90° → (u, v) = (y, −x); frame −90° → (−y, x); 180° → (−x, −y). Subtract the origin first.

Cohen–Sutherland line clipping (P2 Q3, Exam 2020 T3 (4 groups)): Codes by comparison, no arithmetic. Cut points: $(c - y_1)/\Delta y$ as a fraction, then $x = x_1 + \Delta x \cdot \text{fraction}$ (e.g. (5 − 10)/(−7) = 5/7 → x = 8 + 7·5/7 = 13).

Sutherland–Hodgman polygon clipping (P2 Q4): Same cut formula with fractions; 55/9 = 6.11 by long division (9·6 = 54, remainder 1 → 0.11).

Bézier curve construction (+ smooth joining) (P2 Q5, EXAM 1/22 #1): t = ½ is just midpoints (averages). t = ¼, ¾: quarter points (3A + B)/4 and (A + 3B)/4. No multiplication of decimals needed.

k · √2/2 (rotation by 45°)

k	k·0.7071
1	0.7071
2	1.4142
3	2.1213
4	2.8284
5	3.5355
6	4.2426
7	4.9497
8	5.6569
9	6.364
10	7.0711
0.5	0.3536
1.5	1.0607
2.5	1.7678

16:9 diagonal → width, height (inches)

diag	width	height
13″	11.33	6.373
14″	12.202	6.864
15.6″	13.597	7.648
17″	14.817	8.334
21″	18.303	10.295
22″	19.175	10.786
24″	20.918	11.766
27″	23.533	13.237
32″	27.89	15.688
40″	34.863	19.61
42″	36.606	20.591
43″	37.478	21.081
50″	43.579	24.513
55″	47.937	26.964
65″	56.652	31.867

Eye limit: smallest detail x = D·tan 1′ and n = W·(3437.75/D)

D	x	3437.75/D (× W in cm)
25 cm	0.00727 cm	137.51
30 cm	0.00873 cm	114.592
40 cm	0.01164 cm	85.944
50 cm	0.01454 cm	68.755
60 cm	0.01745 cm	57.296
70 cm	0.02036 cm	49.111
80 cm	0.02327 cm	42.972
100 cm	0.02909 cm	34.378
150 cm	0.04363 cm	22.918
200 cm	0.05818 cm	17.189

Fractions → decimals

fraction	value
1/4	0.25
3/4	0.75
1/8	0.125
3/8	0.375
5/8	0.625
7/8	0.875
1/3	0.3333
2/3	0.6667
1/5	0.2
2/5	0.4
3/5	0.6
4/5	0.8
1/6	0.1667
5/6	0.8333
1/7	0.1429
2/7	0.2857
3/7	0.4286
4/7	0.5714
5/7	0.7143
6/7	0.8571
1/9	0.1111
55/9	6.1111

Part D · templates

RGB cube · HSV cone · outcodes · empty pixel grids

